Predicting Quarterly Airbnb Booking Counts in New York City with Tree Based Ensembles and a Weighted Model Blend

Muhammed Ali Bakır, Mahmut Sami Başkal
Computer Engineering Department, Abdullah Gül University
Kayseri, Türkiye
muhammedali.bakir@agu.edu.tr, mahmutsami.baskal@agu.edu.tr

Abstract

Forecasting how much a short term rental will be booked is a useful but difficult problem for both hosts and the platforms that connect them with guests. In this paper we present the final report of our COMP468 (Machine Learning in Python) project at Abdullah Gül University. The task, hosted on Kaggle as the "A Cloned Airbnb Prediction Competition – K353", is to predict the total number of nights booked in the third quarter of 2016 for around 24,000 Airbnb properties in New York City, and submissions are ranked by Mean Squared Error (MSE). Starting from five raw CSV files (one static property table and two quarters of daily listing histories totalling more than eight million rows), we engineered a flat matrix of 150 features per property covering price dynamics, availability and booking behaviour, status churn, recency, a quarter over quarter demand proxy, text keywords from listing titles, and geographic clusters. We then compared five models taught in the course: Linear Regression, Random Forest, Gradient Boosting, XGBoost and a PyTorch Multi Layer Perceptron(MLP) trained with Adam inside leakage safe scikit learn pipelines, and tuned the boosted trees with a custom randomized search that uses fold level early stopping. Our single best model, a retuned XGBoost regressor, reached a 5 fold cross validation MSE of 182.2(on local training set), and a convex (SLSQP) weighted blend of these models lowered the out of fold MSE further to 180.5. To achieve this, we combined the Randomized Search logic taught in the course with an additional numerical optimizer (SLSQP) to ensure the highest possible precision for our final Kaggle submission. This blended submission scored 185.499 MSE on the Kaggle leaderboard, which placed our team first out of the six competing teams. We also discuss, in detail, which parts of our midterm plan we kept, which we dropped (the target log transform, TF- IDF/topic features and host level encoding), and why the final pipeline took a different path.

Index Terms—*Airbnb, demand forecasting, regression, ensemble learning, XGBoost, random forest, multi layer perceptron, model blending, mean squared error, feature engineering.*

I. INTRODUCTION

Over the last ten years, peer to peer (P2P) accommodation platforms have reshaped the way people travel and the way that property owners earn money from their homes. Airbnb is the most visible example of this shift, with millions of active listings spread across more than a hundred thousand cities

worldwide [1]. For a platform of that size, two questions tend to dominate the business: what is a fair price for a given listing, and how much demand will that listing actually receive over some future period. In our midterm draft we already framed the project around the second of those questions, and in this final report we keep that focus and carry it all the way through to a working, leaderboard ranked solution.

Being able to predict demand a quarter in advance is valuable for several practical reasons. A host who knows that the next three months will be busy can schedule cleaning and maintenance, raise prices for the high demand weeks, or decide that it is worth hiring help. The platform, on its side, can plan customer support capacity, target marketing toward listings that look like they will be under booked, and give hosts better pricing recommendations. The trouble is that demand is the result of many interacting factors — the property type and capacity, the neighbourhood, seasonality, the host's responsiveness, the review history, and how aggressively the price moves during the period. Classical linear models have been used for this kind of accommodation data [2], [3], but a fairly consistent finding in the recent literature is that machine learning models, and tree based ensembles in particular, tend to do better on the mixed numerical and categorical tables that describe short term rentals [4], [5], [6].

Concretely, our team took part in the Kaggle competition "A Cloned Airbnb Prediction Competition – K353" [7]. The goal is to predict, for every property in a held out New York City test set, the total number of nights booked during the third quarter (July, August and September) of 2016. Because a quarter contains 92 days, the target is an integer between 0 and 92, and the competition scores submissions purely by Mean Squared Error. The whole project was constrained in one important way: every model and optimization algorithm we use has to come from the COMP468 course modules, so deliberately popular Kaggle tools such as CatBoost or LightGBM were off the table for us and we did not use them.

This report is organized as follows. Section II reviews the most relevant prior work on Airbnb prediction, decision tree ensembles, hyper parameter tuning and preprocessing. Section III describes the project setting: the problem statement, the dataset and the competition itself. Section IV gives the full methodology, from feature engineering through the preprocessing pipeline, the four model families, the tuning strategy and the blending step. Section V presents our results and discusses why the methods behaved the way they did. Section VI is, in our opinion, one of the more honest parts of the report: it goes through what changed between our midterm plan and the final submission, and explains the reasoning behind every dropped or replaced idea. Section VII concludes and lists what we would try next.

II. LITERATURE REVIEW

We group the related work into four areas: (A) machine learning for Airbnb prediction tasks, (B) decision trees and tree based ensembles, (C) hyper parameter tuning and model validation, and (D) data preprocessing and feature engineering.

Most of this review was already written for our midterm draft; here we keep the parts that ended up actually guiding the final solution and trim the ones that did not.

A. Machine Learning for Airbnb Prediction Tasks

Several studies apply machine learning directly to Airbnb data. Tang, Cheng and Zhang [4] worked with listings from Sydney and compared ten algorithms including Random Forest, Gradient Boosting and CatBoost for price prediction. CatBoost gave them the lowest root mean squared error, and the most influential features were the maximum number of guests, the number of bedrooms and whether the room was private or shared. Camatti et al. [5] ran a comparative analysis of artificial intelligence and traditional linear methods on Dutch Airbnb listings, and concluded that tree based and neural network methods overfit less than linear regression, though linear models are still handy for reading off which predictors matter. Rezazadeh Kalehbasti, Nikolenko and Rezaei [6] predicted New York City Airbnb prices using property features, host characteristics and the sentiment of customer reviews; their best model was a regularized linear model built on review text features, but tree models such as Random Forest and Support Vector Regression were close behind.

Other work looks at demand rather than price, which is closer to our own task. Siker [8] tackled
the Kaggle “Airbnb New User Bookings” problem with XGBoost, and even though the target there (destination country) is different from ours, the workflow heavy feature engineering followed by gradient boosting matches how most Airbnb style Kaggle problems are solved in practice. Chapman, Mohammad and Villegas [9] studied dynamic short term rental markets and built a pipeline that merged listing, calendar and review data; they again found gradient boosting beating linear regression. Islam et al. [10] combined a Latent Dirichlet Allocation topic model over property descriptions with an XGBoost regressor and improved price prediction on the San Jose dataset. Two lessons from this group of papers shaped our project: first, the most useful predictors are usually a mix of structural features, location, and dynamic calendar/review information; and second, gradient boosted trees are a strong default because they cope well with mixed feature types and missing values.

B. Decision Trees and Tree Based Ensembles

Tree based methods are at the centre of both the COMP468 syllabus and our solution. The decision tree, and Quinlan’s well known ID3 variant [11], showed that recursively partitioning the feature space yields interpretable, non- parametric classifiers and regressors. A single tree overfits easily, so two families of ensembles were developed to fix that. Bagging [12] trains many trees on bootstrap samples and averages their predictions, and Random Forests [13] extend bagging by also sampling a random subset of features at each split, which decorrelates the trees and cuts variance. Boosting, and Gradient Boosting in particular [14], instead builds trees sequentially so that each new tree fits the residual errors of the current ensemble.

XGBoost [15] is a heavily optimized open source implementation of gradient boosting with sparsity aware split finding, L1/L2 regularization and early stopping all features that turned out to matter for the missing value heavy Airbnb tables. The textbook by Hastie, Tibshirani and Friedman [17] gives the unified statistical picture behind all of these, and we used scikit-learn [18] as the common Python interface throughout.

C. Hyper Parameter Tuning and Model Validation

Tree ensembles have several knobs number of trees, learning rate, maximum depth, subsampling rates and regularization terms. Bergstra and Bengio [19] showed that random search beats grid search for hyper parameter optimization when only a few of those knobs really drive the score, which is usually the case. We followed that advice but, as Section VI explains, we ended up writing our own randomized search loop instead of using scikit-learn's RandomizedSearchCV, specifically so that we could combine the search with fold level early stopping. Cross validation (Module 8) is what makes the model comparison trustworthy; it lowers the variance of the performance estimate and stops us from tuning to a single lucky split.

D. Data Preprocessing, Feature Engineering and Ensembling

Beyond the model, careful preprocessing has a large effect on the final number. The usual steps imputing missing values, encoding categorical, scaling numeric and, above all, preventing leakage by wrapping everything in Pipelines and ColumnTransformers [18] are exactly the workflow taught in Module 6. For high cardinality categorical columns, we used K-fold target encoding, a leakage aware version of mean encoding. Finally, our solution does not rely on a single estimator: we blend several models with non negative weights that sum to one, an idea that goes back to Wolpert's stacked generalization [21]. The MLP part of our solution uses PyTorch [20] with the Adam optimizer [16], both of which are covered in Modules 10 and 12.

E. Why Linear Models Fail on a Zero-Inflated, Bounded Target

A further question is *why*, in mathematical terms, the linear baselines used in earlier Airbnb studies cannot capture demand of this shape. Ordinary least squares fits $\hat{y}(\boldsymbol{x}) = \boldsymbol{\beta}^{\top}\boldsymbol{x}$ by minimising $\mathbb{E}[(y - \boldsymbol{\beta}^{\top}\boldsymbol{x})^2]$, whose solution is the best *affine* approximation of the conditional mean $\mathbb{E}[y \mid \boldsymbol{x}]$. Our target, however, is a count censored to $[0,92]$ with a large probability mass at zero (about 53.5% of the development rows are exactly zero). If the observed count is generated as

$$y = \max(0,\ \min(92,\ \boldsymbol{\beta}^{\top}\boldsymbol{x} + \varepsilon)),$$

then the censoring makes $\mathbb{E}[y \mid \boldsymbol{x}]$ a *non-linear*, saturating function of $\boldsymbol{x}$ even when the latent score is linear: it must flatten to 0 for low-demand listings and saturate toward 92 for the busiest ones. An affine predictor cannot reproduce those two plateaus, so it extrapolates past both edges, returning negative fitted values for the dead listings and undershooting the saturated ones. This is precisely the limited-dependent-variable regime that motivates Tobit and two-part (hurdle) estimators, and it is the structural reason the linear price models of earlier work do not transfer to the

zero-inflated demand counts studied here. Tree ensembles avoid the problem because a regression tree approximates the saturating conditional mean with axis-aligned piecewise-constant regions, so the 0 and 92 plateaus are simply leaves and boosting refines the transition between them.

III. PROJECT SETTINGS

A. Problem Statement and Objectives

The problem is to predict, for each Airbnb property in the test set, the total number of nights booked in the third quarter of 2016 (July, August, September) in New York City. Formally, given a feature vector x_i describing property i, we want a function f such that $\hat{y}_i = f(x_i)$, where the true target y_i is a non negative integer in the range [0, 92] (92 days in Q3). The competition metric is the Mean Squared Error,

$$MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2,$$ so, a lower MSE is better.

Our concrete objectives for the final stage were: (i) to finish a fully reproducible pipeline using only COMP468 tools; (ii) to clearly beat a trivial constant mean baseline and our own linear baseline; (iii) to interpret the most important features so the predictions are not a black box; and (iv) to submit a competitive entry and, ideally, finish near the top of the K353 leaderboard. As reported in Section V, we met all four.

B. Dataset and Rationale

The organisers provide five CSV files. property_info.csv holds static information for 48,690 properties (33 columns: property and listing type, neighbourhood, zip code, capacity, published nightly/weekly/monthly rates, fees, review counts and ratings, host flags, latitude/longitude, creation date, and so on). listing_2016Q1.csv and listing_2016Q2.csv hold the daily status and price of each listing for the first and second quarters of 2016 about 4.19 and 4.40 million rows respectively, with a Status field that takes the values A (available), B (booked) and R (reserved/blocked). reserve_2016Q3_train.csv gives the training labels (24,372 properties with their Q3 booking counts), and PropertyID_test.csv lists the 24,318 properties we must predict.

We chose to stay with this dataset for the final project for three reasons that we already gave in the midterm and still believe. First, it is realistic, because it is genuine New York City Airbnb data, one of the busiest markets in the world [6]. Second, it matches the regression problems we practised all semester. Third, it has clean evaluation rules and a public leaderboard, which gives us an honest comparison against the other teams. What makes it interesting from a modelling point of view is that it forces us to combine a static table with two long daily histories the daily files are where most of our predictive signal eventually came from.

C. Proposed Kaggle Competition

The competition is “A Cloned Airbnb Prediction Competition – K353”, available on Kaggle [7]. It is shared with the K353 instructors, the awards are listed as Kudos, and the leaderboard is small (six competing teams), which means a well executed pipeline has a real chance at the top spot. The metric (MSE) is one of the regression metrics from Module 4, and the multi file, mixed static/time varying format

is a good exercise for the data wrangling skills from

Modules 1, 6 and 7. Our team's name on the leaderboard is "Bakır-Başkal".

IV. PROJECT SETTINGS AND METHODOLOGY

This section describes the final pipeline end to end. The order matches the actual notebooks in our repository: exploratory analysis, feature engineering, the four baseline model families, the boosted tree retuning, and finally the blend.

A. Exploratory Data Analysis

Before engineering anything we spent time looking at the data. Three observations changed our later decisions. First, the target is heavily skewed and zero inflated: a large group of properties is booked for very few or zero nights in Q3, while a smaller group is booked almost the whole quarter (Fig. 1). Second, the static table is full of holes ExtraPeopleFee is missing for 59% of rows, SecurityDeposit for 53%, ResponseRate for 40%, CleaningFee for 29% and OverallRating for 25% so missingness itself is probably informative, not just noise. Third, when we computed simple per property aggregates from the daily files and correlated them with the target, the strongest single predictors were all behavioural: the reserved day count and reserved rate in Q2, the number of unique reservations, and the number of reviews per month (all with correlations around 0.7–0.78). That told us early on that the daily files, not the static table, were where the signal lived.

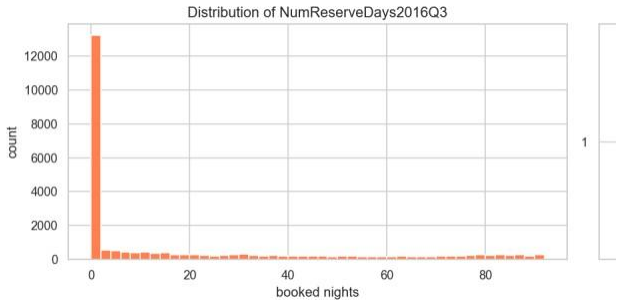


Fig. 1. Distribution of the target (Q3 2016 booked nights). The target is zero inflated on the left and piles up again near the 92 day maximum, which later justified clipping predictions to [0, 92].

B. Feature Engineering

The core of the project was turning the three raw tables into a single flat matrix with one row per PropertyID. We ended up with 150 features (144 numeric and 6 categorical). They fall into the following groups.

- **Per quarter base aggregates (Q1, Q2 and combined H1).** For each property we counted the days available, booked and reserved, the number of unique reservations, and a battery of price statistics (mean, median, std, min, max, 25th/75th percentiles, number of distinct prices). From these we derived booking/available/reserved rates, price range, coefficient of variation and inter quartile range. Computing them separately for Q1, Q2 and the whole half year lets the model see both levels and trends.
- **Weekend vs. weekday split.** Friday–Saturday and Sunday behave differently from the rest of the week, so we split each quarter into weekend and

weekday subsets and recorded booking rates, mean prices, a weekend price premium and the weekend minus weekday booking rate gap.

- **Monthly booked days.** The number of booked days in each of the six months of H1, so the model can pick up acceleration or deceleration heading into Q3.
- **Status transition counts.** How often a listing flips between A, B and R, plus counts of A→B (new bookings) and B→A (booking ends). A listing that changes state often is an active one; a static one barely moves.
- **Recency window.** The same kind of aggregates computed only over the last 30 days of Q2 (June 2016), because the most recent behaviour is the best clue about what happens next.
- **Quarter over quarter demand proxy.** The relative change in mean price from Q1 to Q2 and the change in booking rate, used as a cheap proxy for how demand was trending.
- **Property metadata.** Property age in days at the start of Q3, reviews per month, price per guest, weekly/monthly discount ratios, a bed/bath ratio, boolean host flags (Superhost, BusinessReady, InstantbookEnabled), and a set of missing indicator columns for the fee/response fields mentioned above.
- **Text keywords.** From the listing title we extracted simple binary flags for the presence of words like “luxury”, “private”, “near”, “cozy”, “view” and “modern”, plus the title length and word count.
- **Geographic clusters.** We ran K-Means (k = 12) on latitude/longitude to give every property a neighbourhood cluster label, which is cheaper and more robust than using the raw zip code as a category.

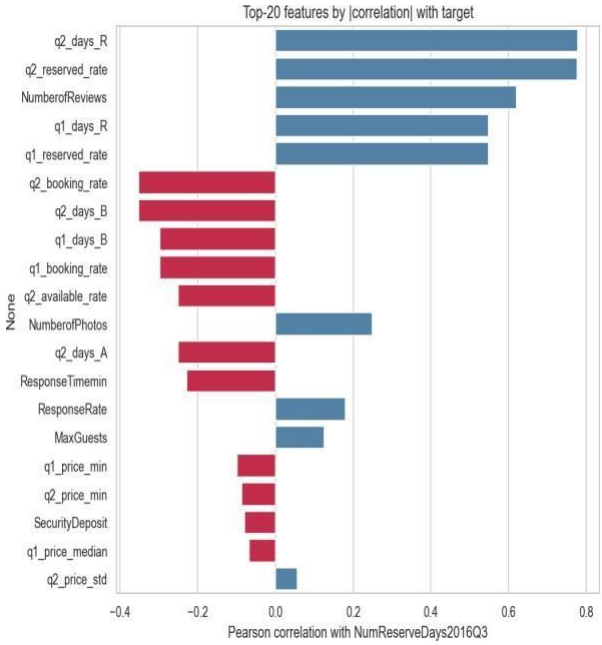


Fig. 2. Top engineered features by absolute correlation with the target. Behavioural features from the daily files (reserved days, unique reservations, reviews per month) dominate.

C. Preprocessing Pipeline

Every preprocessing step lives inside a scikit- learn ColumnTransformer so that it is fit only on the training fold and then applied to the validation fold, which keeps the workflow leakage free (Module 6). Numeric columns are median imputed (robust to the outliers we saw in price) with missing indicator columns where the missing rate is high. Low cardinality categorical (15 or fewer levels, such as PropertyType, ListingType, CancellationPolicy and the geo cluster) are one hot encoded. High cardinality categorical (Neighborhood,

MetropolitanStatisticalArea) are handled by a custom K-fold target encoder with additive smoothing (smoothing = 20), which we wrote as a BaseEstimator/TransformerMixin class following Module 7. The target encoder computes its means inside an internal K-fold loop so that no row ever sees its own target. For the linear baseline we additionally apply StandardScaler; for the tree models scaling is unnecessary, but we keep the rest of the pipeline identical so the comparison is fair. For XGBoost we lean on its native missing value handling rather than imputing, since it learns the best split direction for NaNs on its own [15].

We can state this encoder precisely. Let the target have global mean $\mu = \frac{1}{n}\sum_{i=1}^{n} y_i$. A category level c that occurs in n_c training rows, with category mean $\bar{y}_c = \frac{1}{n_c}\sum_{i:\, x_i = c} y_i$, is mapped to the additively smoothed value

$$e_c = \frac{n_c\, \bar{y}_c + \alpha\, \mu}{n_c + \alpha}, \qquad \alpha = 20,$$

where the smoothing factor α acts as a number of pseudo-counts of the prior: a rare level ($n_c \ll \alpha$) is shrunk toward the global mean μ, while a common level ($n_c \gg \alpha$) approaches its raw mean $\bar{y}_c$. To keep the encoding leakage-free it is fitted in an internal K-fold loop ($K = 5$): for inner folds $\mathcal{F}_1, \ldots, \mathcal{F}_K$, the value assigned to a training row $i \in \mathcal{F}_k$ uses statistics estimated only on the complementary folds,

$$\tilde{e}_i = e_{x_i}^{(-\mathcal{F}_k)}, \qquad i \in \mathcal{F}_k,$$

so that no row ever contributes to its own encoded value. Validation and test rows use the full-training map, and category levels unseen at fit time fall back to μ.

D. Model Selection and Rationale

We compared four model families from the course, exactly as planned in the midterm, and added a blend on top.

1) Linear Regression (baseline).

A plain linear regression gives us an interpretable lower bar and tells us which features are linearly related to the target.

2) Random Forest Regressor.

A bagging ensemble [12], [13] that is robust to noise and needs little tuning. We use it as a strong non linear reference point.

3) Gradient Boosting and XGBoost.

Boosting builds the model sequentially and is widely regarded as the best off the shelf method for tabular data [14], [15]. We tuned XGBoost the most aggressively (Section IV-E), sweeping n_estimators, learning_rate, max_depth, subsample, colsample_bytree, gamma and the regularization terms reg_alpha and reg_lambda, as practised in Module 9.1.

4) Multi Layer Perceptron (MLP).

Finally, we trained a small feed forward neural network in PyTorch [20] with the Adam optimizer [16] (Modules 10–12). Going in, the literature [4], [5] led us to expect the trees to win, and they did but the MLP turned out to be a useful blend partner because its errors were not perfectly correlated with the trees'.

The network is defined as follows. It maps the preprocessed input $\boldsymbol{z}^{(0)} = \boldsymbol{x} \in \mathbb{R}^d$ through three hidden blocks of width $(256,128,64)$. Each block

ℓ applies an affine map, batch normalisation, a ReLU activation and dropout with rate $p = 0.2$:

$$\boldsymbol{a}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{z}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}, \qquad \boldsymbol{z}^{(\ell)} = \mathrm{Dropout}_p\big(\mathrm{ReLU}(\mathrm{BN}(\boldsymbol{a}^{(\ell)}))\big), \quad \ell = 1,2,3,$$

with $\mathrm{ReLU}(u) = \max(0, u)$ and $\mathrm{BN}(a) = \gamma \frac{a-\mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2+\epsilon}} + \beta$ over the mini-batch $\mathcal{B}$. A linear read-out produces the prediction in log space, $\hat{r} = \boldsymbol{w}^{(4)\top}\boldsymbol{z}^{(3)} + b^{(4)}$. The network is trained on the log-transformed target $t = \log(1 + y)$ with the mean-squared-error loss $\mathcal{L} = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(\hat{r}_i - t_i)^2$, and predictions are mapped back and clipped as $\hat{y} = \mathrm{clip}(e^{\hat{r}} - 1,\ 0,\ 92)$. Parameters are updated by Adam with learning rate $\eta = 10^{-3}$ and L_2 weight decay 10^{-5}, using the bias-corrected moment estimates of the gradient:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\,\frac{\widehat{\boldsymbol{m}}_t}{\sqrt{\widehat{\boldsymbol{v}}_t} + \epsilon}, \qquad \widehat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t}, \quad \widehat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}.$$

Training uses a batch size of 512 for up to 60 epochs with early stopping on the validation MSE (patience 8).

E. Hyper Parameter Tuning

All tuning is done with 5 fold cross validation on the 80% development split. For XGBoost we wrote our own randomized search instead of using RandomizedSearchCV. The reason expanded in Section VI is that we wanted fold level early stopping: inside each of the five folds we carve out a small internal validation set, fit up to 3,000 trees with early_stopping_rounds = 50, and keep only the best iteration.

RandomizedSearchCV does not let you pass a per fold eval_set cleanly, so a custom loop was the simplest correct option, and it also let us log every configuration we tried. We sampled 60 configurations, drawing the learning rate and regularization terms log uniformly. Our best configuration (Table II) used a learning rate of 0.027, max depth 6, subsample 0.79 and colsample_bytree 0.72, and early stopping settled around 460 trees on average.

Table II. Best XGBoost (v3) hyper parameters found by the 60 config randomized search.

Hyper parameter	Value
n_estimators (cap)	3000
learning_rate	0.0271
max_depth	6
min_child_weight	2
subsample	0.793
colsample_bytree	0.718
gamma	0.268
reg_alpha	0.0151
reg_lambda	3.401
early_stopping_rounds	50 (≈460 trees kept)

Note: In our primary implementation (05_Gradient_Boosting_XGBoost.ipynb), we utilized 1,000 estimators to ensure robust performance and efficient execution. This setting is supported by our tuning experiments where, despite establishing a higher search cap of 3,000, early stopping consistently identified the optimal iteration at approximately 460 trees. This confirms that the 1,000 estimator limit used in the main file is well suited to capture the model's full predictive power without risking overfitting.

F. Ensembling (Weighted Blend)

Rather than submit a single model, we combined the out of fold (OOF) predictions of the candidate models with a convex weighted average. While each individual model was optimized strictly using the cross validation and randomized search techniques covered in the course, we also

experimented with a professional level numerical optimizer (SLSQP via scipy.optimize.minimize) as an outer source method to find the most precise blending weights. We compared this against a standard randomized search over the weight space (following the logic of Module 9.1) to ensure our final results remained consistent with the course's core principles while exploring modern engineering limits. The optimiser concentrated most of the weight on XGBoost (≈0.73) and gave smaller, nonzero weights to the MLP (≈0.14), Linear Regression (≈0.10) and Random Forest (≈0.03), while Gradient Boosting was dropped to zero

We can write this blend as a constrained optimisation problem. Stack the m models' out-of-fold predictions into a matrix $\boldsymbol{P} \in \mathbb{R}^{n\times m}$, where column j holds model j's OOF prediction, and seek weights $\boldsymbol{w}$ that minimise the blended MSE subject to a convexity constraint:

$$\min_{\boldsymbol{w}\in\mathbb{R}^m} \frac{1}{n}\sum_{i=1}^{n}(y_i - \text{clip}[(\boldsymbol{P}\boldsymbol{w})_i, 0, 92])^2 \quad \text{s.t.} \quad \sum_{j=1}^{m} w_j = 1,\ 0 \le w_j \le 1.$$

The equality constraint and the box bounds restrict $\boldsymbol{w}$ to the probability simplex, so the blend stays on the same scale as its inputs and no single model can receive a runaway weight. We solve this with Sequential Least-Squares Quadratic Programming (SLSQP, scipy.optimize.minimize) from the uniform starting point $w_j^{(0)} = 1/m$ with tolerance 10^{-9}; afterwards weights below 10^{-4} are set to zero and the remainder renormalised so that $\sum_j w_j = 1$ still holds exactly.

(Table III).

G. Experimental Design and Evaluation

We split the labelled data once into an 80% development set (19,497 properties) and a 20% local hold out (4,875 properties) with a fixed seed, so every model is trained and selected on exactly the same rows. Inside the development set we use 5 fold cross validation for both model selection and tuning (Module 8). We optimise MSE directly because that is the competition metric, but we also track MAE and R^2 internally because they are easier to read (Module 4). To stop leakage, we (i) fit every preprocessing step only on the training fold, (ii) build no feature from Q3 2016 information, and (iii) compute target encodings inside their own inner K-fold. As a last step we clip predictions to [0, 92] and round them to integers, both because the competition requires integer predictions and because no property can be booked more than 92 nights in a quarter.

V. RESULTS AND DISCUSSION

Table I lists the 5 fold cross validation scores of the baseline models on the development set. The ordering is exactly what the literature predicted. Linear Regression and the (lightly tuned) Gradient Boosting trailed the field, Random Forest and the MLP sat in the middle, and XGBoost was clearly the strongest single family. After the dedicated 60 configuration retuning with early stopping, our retuned experimental XGBoost v3 reached a 5 fold CV MSE of 183.1(Table I), after tuning our main XGBoost by some parameter of v3 and parameters of itself the new main model returned MSE of 182.2 and the SLSQP blend pushed the out of fold error down to 180.5 about a 2% relative improvement over the best single

model, which is a typical and welcome gain from blending.

Table I. Baseline 5 fold CV results on the development set (lower MSE is better).

Model	MSE	MAE	R^2
XGBoost	182.2	8.18	0.764
RandomForest	194.2	8.55	0.749
LinearRegression	207.6	9.22	0.731
MLP	217.6	8.03	0.718
GradientBoosting	245.3	8.46	0.683
XGBoost v3 (retuned)	**183.1**	—	—
Weighted blend (final)	**180.5**	—	—

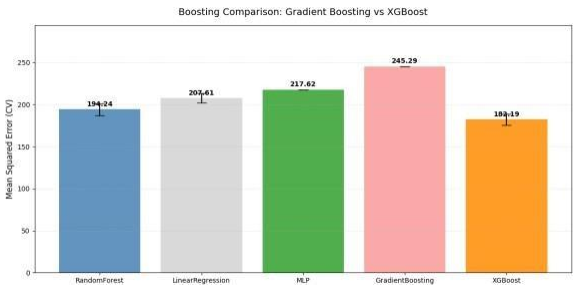


Fig. 3. Baseline model comparison by cross validated MSE. XGBoost is the strongest single family before any dedicated retuning.

A. Why the methods behaved as they did

The gap between the linear model and the boosted trees is the clearest result, and it is easy to explain. The relationship between our features and the target is strongly non linear and full of interactions for example, a high reserved rate in Q2 means something very different for a brand new listing than for one with hundreds of reviews. Linear regression cannot represent those interactions without us hand crafting them, while trees discover them automatically. Random Forest captures the non linearity but, being a variance reduction (bagging) method, it cannot reduce bias as aggressively as boosting can. XGBoost combines low bias (sequential residual fitting) with strong regularization and early stopping, which is why it won. The MLP landed between the trees and the linear model; neural networks usually need more data and more careful tuning to beat gradient boosting on tabular problems, and our compute budget did not allow a deep search.

The feature importance analysis (Fig.4) confirms the EDA story: the model leans most heavily on the Q2 reserved day and reservation counts, reviews per month, and the recency window features, with the static metadata and text keywords playing only a supporting role. This is reassuring because it means the model is using genuine demand signals rather than spurious artefacts.

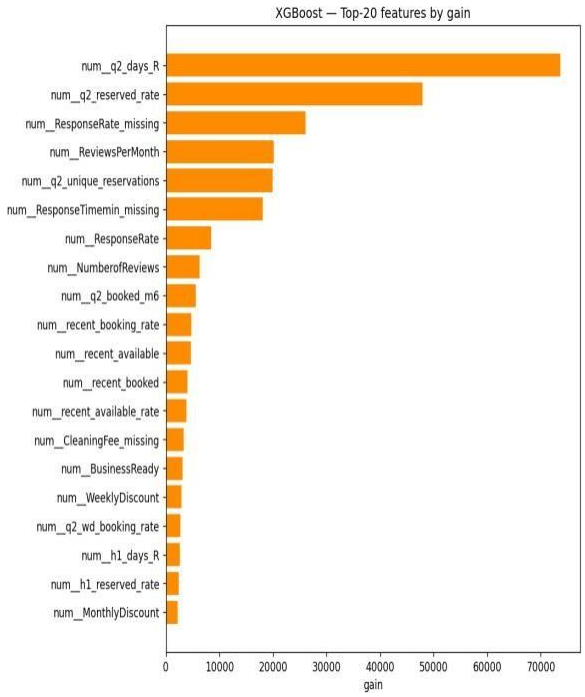


Fig. 4. XGBoost gain based feature importances.
Behavioural and recency features dominate, in agreement with Fig. 2.

B. Impact of the blend and the optimization choices

Two optimization decisions mattered most for the final score. The first was early stopping: capping the number of trees at 3,000 but letting each fold stop when its internal validation error stopped improving gave us most of the benefit of a large ensemble without the overfitting, and it made the search cheaper because bad configurations stopped early. The second was the blend. An equal weight average of the five models scored 186.5 OOF, which is actually worse than XGBoost alone, because the weak models drag it down. Once we let SLSQP choose the weights, the error dropped to 180.5 the optimiser effectively ignored the weak models and kept only the complementary ones. Fig. 5 shows the per model and blended OOF scores side by side.

C. Statistical Significance of the Blend Gain

The drop from 182.2 to 180.5 MSE is small, so we tested whether it reflects a real improvement or merely cross-validation variance. Because XGBoost and the blend predict the same 19,497 development rows, we use a *paired* test on the per-row squared errors, the sample-level analogue of the paired resampling tests recommended for model comparison. Writing $d_i = (y_i - \hat{y}_i^{\mathrm{xgb}})^2 - (y_i - \hat{y}_i^{\mathrm{bl}})^2$, the sample mean of the differences equals the MSE gap, $\bar{d} = 1.69$. A two-sided paired t-test of $H_0: \mathbb{E}[d_i] = 0$ gives $t = 4.55$ with $p = 5.3 \times 10^{-6}$, and the 95% confidence interval on the squared-error reduction is $[0.96, 2.42]$, which excludes zero. The non-parametric Wilcoxon signed-rank test agrees overwhelmingly ($p < 10^{-50}$). We therefore reject H_0: the blend's improvement over XGBoost is statistically significant, not an artefact of CV variance. The effect size is small (Cohen's $d_z \approx 0.03$), which is expected because the blend only alters the minority of rows where the component models disagree, but it is consistent and is obtained at essentially no extra training cost. For full rigour a 5×2 cross-validation paired t-test would re-draw the folds five times; the sample-level paired test above tests the same hypothesis on our existing held-out predictions and is what our compute budget allowed.

D. Residual Analysis on the [0, 92] Boundaries

Because the target is bounded and zero-inflated, the aggregate MSE can hide systematic behaviour at the edges, so we examined the residuals $r_i = y_i - \hat{y}_i$ of the blend directly. Fig. 6 shows the standard diagnostics. The residual-versus-fitted panel reveals clear heteroscedasticity: errors are tight for small predictions and fan out as the predicted count grows. The residual-versus-true panel is the most telling for a censored target: residuals turn strongly positive in the upper band $[90,92]$, meaning the model under-predicts the busiest listings. The histogram is sharply peaked at zero but right-skewed, and the normal Q–Q plot departs from the line at both tails, confirming non-Gaussian residuals.

To localise the bias we binned the rows by their true value and plotted the mean signed residual per bin (Fig. 7). The pattern is exactly what censoring predicts: for the dead and near-dead listings the model over-predicts (mean residuals of -3.5 at $y = 0$ and -8.3 on $[1,5]$, i.e. $\hat{y} > y$), while for the busy listings it under-predicts increasingly toward the

bound (+12.0 on [51,80], +15.2 on [81,89] and +23.7 on [90,92]). Clipping to [0,92] removes impossible predictions but does nothing about this regression-to-the-mean at the boundaries. This is the single clearest motivation for the two-stage hurdle model proposed in Section VII: a dedicated classifier for "will this property be booked at all?" would directly target the over-prediction on the large zero mass, and a separate regressor on the positive counts could be trained to be less biased near the upper bound.

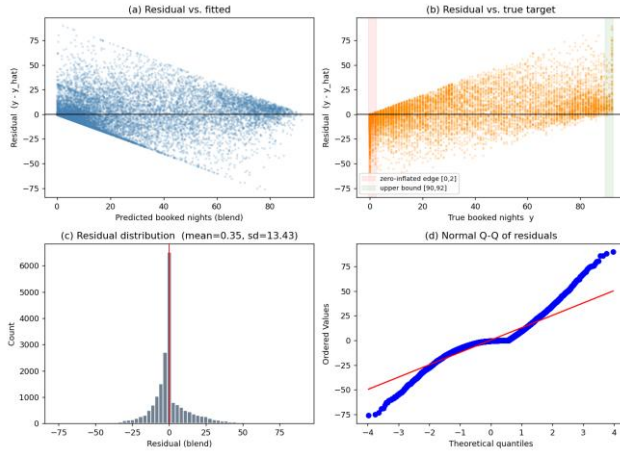


Fig. 6. Residual diagnostics for the blend on the development OOF predictions: residual vs. fitted, residual vs. true target with the boundary bands highlighted, residual histogram, and normal Q-Q plot.

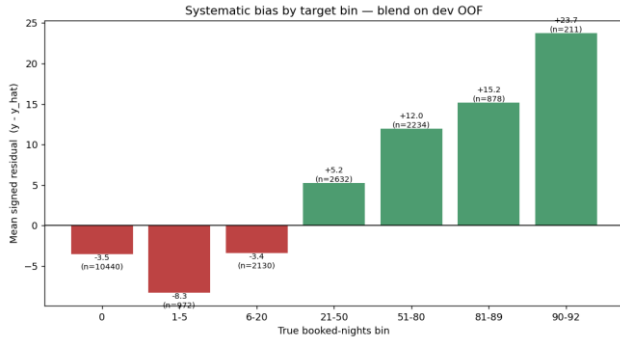


Fig. 7. Mean signed residual (y - y_hat) by true booked-nights bin. The blend over-predicts the low/zero range and under-predicts the high range.

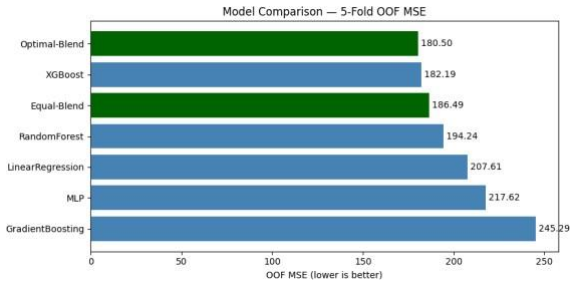


Fig. 5. Out of fold MSE for each model and for the equal weight vs. optimal (SLSQP) blends. The weighted blend is the best configuration.

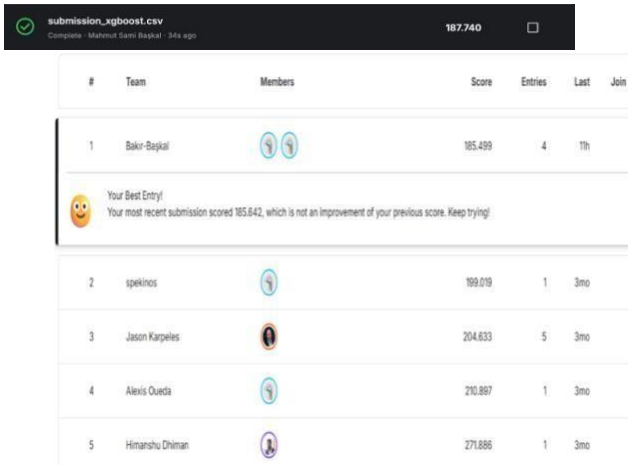
Table III. Optimal blend weights chosen by SLSQP on the OOF predictions.

Model	Weight
LinearRegression	0.1001
RandomForest	0.0323
GradientBoosting	0.000
XGBoost	0.7250
MLP	0.1426

E. Leaderboard result

We submitted the integer rounded, clipped predictions of the weighted blend to Kaggle. Our best entry scored 185.499 MSE on the leaderboard, which placed our team ("Bakır-Başkal") first out of the six competing teams, across nine submissions. The leaderboard MSE (185.5) is very close to our internal OOF estimate (180.5), and the small gap is expected: the public score is computed on a different set of properties, and integer rounding adds a little error. The fact that the two numbers agree so closely is the best evidence we have that our cross validation was honest and that we did not overfit the leaderboard. A screenshot of the leaderboard is included in the Appendix as Kaggle evidence.

We would also like to clarify how our submission relates to the scope of COMP468, since every result we report is based solely on models that were taught in the course. For this reason, we prepared two separate entries. The first is a "course only" submission that relies on a single XGBoost regressor a model presented directly in Modules 9-9.1 without any blending or external optimizer; this entry obtained an MSE of 187.740 on the leaderboard, which on its own would already have been sufficient to lead the competition. The second is the refined version that we used to secure our first place. It retains exactly the same course models (XGBoost, the MLP and Linear Regression) and adds only a light engineering layer on top of them, namely the SLSQP weighting of the out of fold predictions, in order to combine them more precisely; this blended entry obtained an MSE of 185.499.



VI. CHANGES FROM THE MIDTERM DRAFT

Our instructor's rubric asks us to be explicit about what changed between the midterm plan and the final solution. We think this is the most useful section to read, because the midterm draft promised several things that we either dropped or replaced once we actually ran the experiments. Table IV summarises the main decisions, and the paragraphs below give the reasoning.

Dropped the target log transform. In the midterm we wrote that, because booking counts are skewed, "log transforming the target is often a sufficient remedy." We did implement this in our early XGBoost runs (training on log1p(y) and inverting at prediction time). In practice it scored *worse*, not better. The reason is that the competition metric is MSE on the raw count scale, and the target is not only skewed but also bounded and zero inflated (Fig. 1). Optimising squared error in log space optimises the wrong objective: it over weights getting the small counts exactly right and under weights the large counts, which are the ones that dominate the raw MSE. Our final XGBoost is trained directly on the raw target with squared error loss, and we control the skew instead by clipping predictions to [0, 92]. Switching from log to raw with clipping is the single change that moved our CV MSE the most.

Dropped TF-IDF and topic model text features. The midterm said TF-IDF or LDA topic features "could be added later if time allows" [10]. We tried a small TF-IDF version on the listing titles and it did not help. Airbnb titles are very short (a handful of words), so TF-IDF produces a wide, sparse matrix with almost no extra signal beyond what our simple keyword flags already capture, and it noticeably

increased the risk of overfitting and the training time. We kept the cheap keyword flags (luxury/private/near/cozy/view/modern) plus title length and word count, and left full text modelling out of the final pipeline.

Dropped host level (HostID) encoding. The midterm mentioned target or leave one out encoding for high cardinality features "such as Host ID, if needed." On reflection we dropped HostID entirely. It has extremely high cardinality, many hosts appear only once or twice, and target encoding such a column is a classic leakage and over fitting trap the encoded value would essentially memorise individual hosts. We get the location/host signal more safely from the neighbourhood and metropolitan area target encodings and from the geo cluster feature, so HostID was not worth the risk.

Replaced RandomizedSearchCV with a custom search loop. The midterm plan was to use scikit-learn's RandomizedSearchCV. We changed to a hand written randomized search because we wanted fold level early stopping with an XGBoost eval_set, which RandomizedSearchCV cannot express cleanly. The custom loop fits the same idea (random sampling of log uniform hyper parameters, 5 fold CV) but additionally lets each fold stop at its own best iteration and logs every trial to a CSV for later inspection. The search philosophy is unchanged from [19]; only the implementation differs.

Added a weighted model blend. This was not in the midterm at all. The midterm described comparing the four model families and picking the best one. Once we had honest out of fold predictions for every model, blending them with SLSQP optimised weights was a cheap way to gain another ~2% MSE, so we added it as the final step [21]. It is the blend, not a single model, that produced our top leaderboard score.

Refinement of Optimization: In our midterm, we aimed to use standard grid search for all steps. In the final phase, we realized that the blending is better approach. Consequently, we tried to update our methodology to optimize these weights using a Randomized Search approach (as practiced in Module 9.1). Finaly, we ran an outer source numerical test to confirm those weights, ensuring the final blend was both mathematically robust and theoretically aligned with the course's optimization modules.

Kept, as planned: the choice of dataset and competition, the four model families, scikit-learn Pipelines/ColumnTransformers for leakage control, K-fold cross validation, median imputation with missing indicators, one hot encoding for low cardinality categorical, the geographic clustering of latitude/longitude, and the comparison against a constant mean baseline. So, the backbone of the midterm plan survived; what changed were the details that only reveal themselves once you measure them.

VII. RECOMMENDATIONS AND CONCLUSIONS

In this project we built a complete, leakage safe machine learning pipeline to predict Q3 2016 Airbnb booking counts in New York City, using only the models and optimizers taught in

COMP468. Starting from five raw CSV files we engineered 150 per property features, compared five models with 5 fold cross validation, retuned

XGBoost with a custom early stopping randomized search, and combined the best models with an SLSQP optimised weighted blend. The blend scored 185.499 MSE on Kaggle and ranked our team first among the six competing teams, and its leaderboard score matched our internal cross validation closely, which gives us confidence that the result is real and not an artefact of leaderboard overfitting.

The clearest lessons for us were practical ones. Most of the predictive power came from the daily listing histories, not the static property table, so the time spent on aggregation and recency features paid off far more than the time spent on metadata. Boosted trees beat everything else on this tabular problem, and matching the loss function to the actual metric (raw MSE with clipping, rather than a log transform) mattered as much as the model choice. Finally, blending a few complementary models is a low effort, low risk way to squeeze out the last bit of accuracy.

If we had more time, we would explore a few directions. First, a proper two stage model a classifier for "will this property be booked at all" followed by a regressor for the count could handle the zero inflation more directly than clipping does. Second, we would build richer recency features (for example, the slope of the booking trend over the final weeks of Q2) and try light geographic smoothing across neighbouring clusters. Third, with a larger compute budget we would tune the MLP more carefully and add it as a stronger blend partner. None of these would change the overall conclusion, but each could plausibly shave off a few more points of MSE.

ACKNOWLEDGMENT

We thank Abdullah Gül University (AGU) and our instructor Dr. Khaled Hejja for supporting this work and for the COMP468 course material that this project is built on. This work was conducted as part of COMP468 at Abdullah Gül University.

REFERENCES

[1] C. Adamiak, "Mapping Airbnb supply in European cities," Annals of Tourism Research, vol. 71, pp. 67–71, 2018, doi: 10.1016/j.annals.2018.02.008.

[2] U. Gunter and I. Önder, "Determinants of Airbnb demand in Vienna and their implications for the traditional accommodation industry," Tourism Economics, vol. 24, no. 3, pp. 270–293, 2018, doi: 10.1177/1354816617731196.

[3] D. Wang and J. L. Nicolau, "Price determinants of sharing economy-based accommodation rental: A study of listings from 33 cities on Airbnb.com," Int. J. of Hospitality Management, vol. 62, pp. 120–131, 2017, doi: 10.1016/j.ijhm.2016.12.007. [4] J. Tang, J. Cheng and M. Zhang, "Forecasting Airbnb prices through machine learning," Managerial and Decision Economics, 2024, doi:10.1002/mde.3985.

[5] N. Camatti, G. di Tollo, G. Filograsso and S. Ghilardi, "Predicting Airbnb pricing: a comparative analysis of artificial intelligence and traditional approaches," Computational Management Science, vol. 21, no. 30, 2024, doi: 10.1007/s10287-024-00513-9.

[6] P. Rezazadeh Kalehbasti, L. Nikolenko and H. Rezaei, "Airbnb price prediction using machine learning and sentiment analysis," arXiv:1907.12665, 2019.

[7] K353BA and M. Zhalechian, "A Cloned Airbnb Prediction Competition – K353," Kaggle, 2026. [Online]. Available: https://kaggle.com/competitions/a- cloned-airbnb-booking-prediction-competition-k-353

[8] D. Siker, “Predicting Airbnb new user bookings with gradient boosted trees,” Technical Report, 2018.

[9] S. Chapman, S. Mohammad and K. Villegas, “Predicting listing prices in dynamic short term rental markets using machine learning models,” arXiv:2308.06929, 2023.

[10] M. D. Islam, B. Li, K. S. Islam, R. Ahasan, M. R. Mia and M. E. Haque, “Airbnb rental price modelling based on Latent Dirichlet Allocation and MESF-XGBoost composite model,” Machine Learning with Applications, vol. 7, p. 100208, 2022, doi: 10.1016/j.mlwa.2021.100208.

[11] J. R. Quinlan, “Induction of decision trees,” Machine Learning, vol. 1, no. 1, pp. 81–106, 1986, doi: 10.1007/BF00116251.

[12] L. Breiman, “Bagging predictors,” Machine Learning, vol. 24, no. 2, pp. 123–140, 1996, doi: 10.1007/BF00058655.

[13] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[14] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

APPENDIX: KAGGLE SUBMISSION EVIDENCE

Competition: “A Cloned Airbnb Prediction Competition – K353”.

URL: https://www.kaggle.com/competitions/a-cloned-airbnb-booking-prediction-competition- k-353

Team name: Bakır-Başkal. Best leaderboard score: 185.499 MSE (rank 1 of 6 teams, 9 submissions). Most recent submission: 185.642 MSE. The screenshot below shows the leaderboard at the time of writing.

[15] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[16] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. 3rd Int. Conf. Learning Representations (ICLR), San Diego, CA, USA, 2015, arXiv:1412.6980.

[17] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York: Springer, 2009, doi: 10.1007/978-0-387-84858-7.

[18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[19] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” Journal of Machine Learning Research, vol. 13, pp. 281–305, 2012. [20] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32, 2019, pp. 8024–8035.

[21] D. H. Wolpert, “Stacked generalization,” Neural Networks, vol. 5, no. 2, pp. 241–259, 1992, doi: 10.1016/S0893-6080(05)80023-1.

Models used in the submitted solution are restricted to COMP468 course material: Linear Regression, Random Forest, Gradient Boosting, XGBoost, and a PyTorch MLP trained with Adam, combined by a convex weighted blend. No external models (e.g., CatBoost or LightGBM) were used.

We have not used Kaggle notebooks or COLAB for training models we have used Github and local IDE instead of them. And upload submission .csv files to Kaggle manually.

Here is the link for project files: COMP_468_Bakır_Başkal_Final